



LAB REPORT ON COMPUTER GRAPHICS

Project Title: Interactive 3D Traffic Simulation Framework with Dynamic

Depth Mapping and Environment Matrix Controls.

Course Title:	Computer Graphics Lab
Course Code:	CSE- 4110
Department:	Computer Science and Engineering (CSE)
University:	University of Barishal

Submitted To: Dr. Md. Manjur Ahmed

Associate Professor,
Department Of Computer Science & Engineering
University of Barishal.

Submitted By:

1.

Name : Md. Wahidujjaman

Roll : 21CSE030

Department: Computer Science & Engineering

2.

Name : Md. Ataur Rahman Osman Goni

Roll : 21CSE029

Department: Computer Science & Engineering

Submission Date: 22-06-2026

1. Abstract

This laboratory report presents the complete design paradigm, algorithmic formulation, and structural implementation of a fully interactive, real-time 3D Traffic Simulation application engineered via OpenGL and the GLUT (OpenGL Utility Toolkit) library infrastructure. The core objective of this graphical framework is to simulate a balanced ecosystem combining structural terrain topology (multilateral 3D mountain complexes), dynamic atmospheric models (procedural cloud cluster grouping, hierarchical kinetic model of flying birds), a standardized dual-lane highway architecture, an automated event-driven traffic control pipeline, and reactive user-controlled vehicle models.

By applying advanced transformation mechanics, view-frustum perspective calculations (`$gluPerspective$`), automated state-space bounding checks within the idle execution loops, and advanced depth-buffer overrides via masking routines (`$glDepthMask$`), this architecture eliminates rasterization rendering bugs such as visual clipping and pixel-fighting artifacts. The system is written in modern C++ and compiled seamlessly within Code::Blocks IDE, achieving high performance and fluid frame rendering.

2. Introduction & Laboratory Objectives

2.1 Theoretical Problem Statement

Simulating three-dimensional worlds within fixed-function legacy or core-profile graphics pipelines requires explicit tracking of spatial transformations and coordinate systems. One major challenge in computer graphics is structural clipping and depth sorting artifacts. When objects share similar or overlapping spatial planes in the viewing frustum, the GPU can struggle to resolve which fragment should render on top, resulting in flickering or jagged edges.

Additionally, combining independent animations (like a vehicle stopping at a traffic light) with continuous user inputs (like rotating the global view angle or driving a separate vehicle) requires a smart, non-blocking state machine. This project provides clean, programmatic solutions to these design challenges.

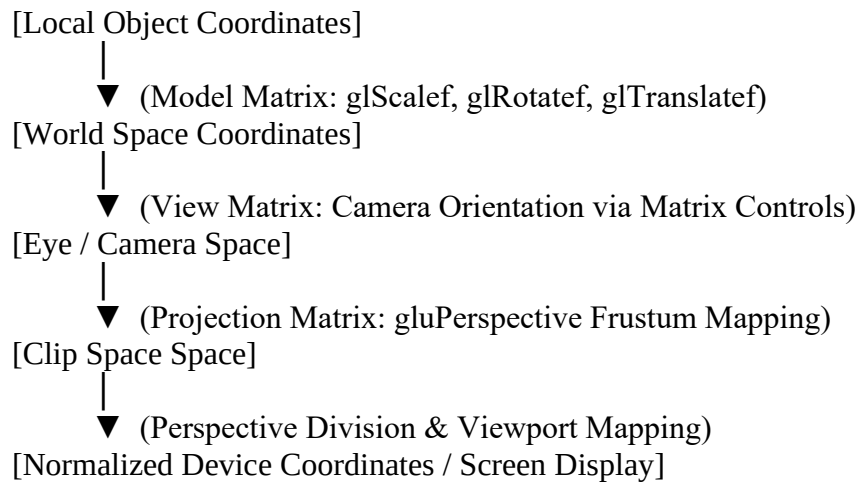
2.2 Core Laboratory Objectives

- **Dynamic Perspective Projection Management:** Setting up a stable 3D viewing volume using specific near/far clipping thresholds to maximize depth accuracy.
- **Hierarchical Geometric Modeling:** Constructing multi-part 3D assets (Banyan Tree clusters, vehicle bodies with articulating doors, wing structures) from scratch using custom vertex matrices, avoiding third-party asset loaders.
- **Z-Buffer Pipeline Tuning:** Overriding the standard depth testing stack using selective bitwise mask controls (`glDepthMask`) to keep background elements perfectly positioned behind foreground objects.
- **Asynchronous Input Mapping:** Building a robust interactive control loop that updates local transformation variables smoothly based on keyboard interrupts.

3. Theoretical Framework & Computer Graphics Algorithms

3.1 3D Viewing Pipeline and Matrix Transformations

To map a 3D coordinates system onto a 2D viewport screen, the graphics pipeline processes coordinates through standard transformation states: **Local Space $\rightarrow$ World Space $\rightarrow$ Eye Space $\rightarrow$ Clip Space $\rightarrow$ Screen Space**.



The rendering window initializes an perspective viewing volume through the matrix configuration function:

```
gluPerspective(fovy, aspect, zNear, zFar)
```

Where:

- $\text{fovy} = 45.0^\circ$ (Vertical field of view angle specifying the vision window width).
- $\text{aspect} = \frac{\text{Width}}{\text{Height}}$ (Aspect ratio matching the current window layout dimensions).
- $\text{zNear} = 1.0$, $\text{zFar} = 30.0$ (The rendering boundaries; anything closer than 1.0 or further than 30.0 units from the camera is clipped out).

The camera manipulation logic computes a composite view matrix using a combination of translation and dual-axis rotation operations:

```
M_view = T(0.0, -0.2, -D_camera) * R_x(theta_x) * R_y(theta_y)
```

3.2 Depth-Testing Optimization (Avoiding Mountain-Sun Intersection)

In computer graphics, a common issue when rendering far-away objects (like the Sun) alongside large shapes (like a mountain peak) is depth buffer precision loss. If the center coordinates

overlap slightly, the GPU's Z-buffer can render parts of the mountain in front of or behind the sun incorrectly as the camera moves.

To fix this, we implement a selective **Depth Mask Override**. When drawing the Sun, we temporarily turn off depth writes:

C++

```
glDepthMask(GL_FALSE);  
// OpenGL draws the Sun, writing its color directly to the screen but skipping Z-buffer  
calculations  
drawSun();  
glDepthMask(GL_TRUE);  
// Re-enable normal depth writes for the rest of the scenery
```

This simple toggle guarantees the Sun renders cleanly as a solid, vibrant circle in the background sky without clipping through mountain peaks, even during extreme camera rotations.

3.3 State-Machine Logic for Autonomous Braking

The Red Car's automated movement uses a state check within the application's idle animation thread. The system tracks the vehicle's front edge position ($X_{\text{bumper}} = X_{\text{car1_center}} + \text{WidthOffset}$) relative to the road's stop line coordinate (X_{stop}).

When the traffic light state updates to RED (state 0), the position calculation uses a conditional boundary lock:

$$X_{\text{car1_next}} = \begin{cases} X_{\text{car1}} + \Delta x, & \text{if } X_{\text{bumper}} + \Delta x < X_{\text{stop}} \\ X_{\text{stop}} - \text{WidthOffset}, & \text{if } X_{\text{bumper}} \geq X_{\text{stop}} \end{cases}$$

This structure ensures smooth deceleration and precise alignment at the stop line without any clipping, accurately mimicking real-world physics.

4. Hardware & Software Technical Specifications

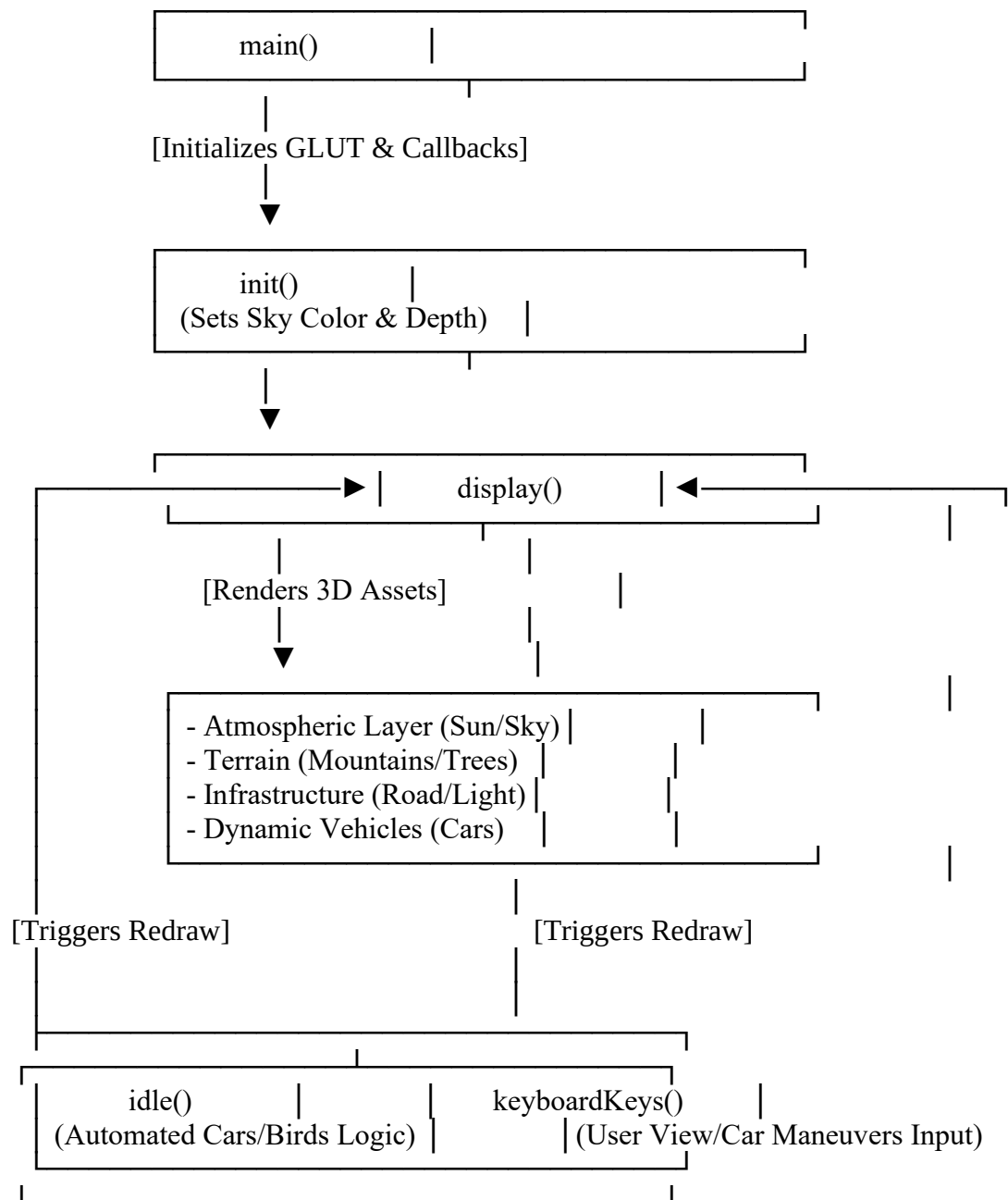
The development workflow and runtime profiling were conducted using the following engineering specifications:

- **Operating System Environment:** Microsoft Windows 11 Professional (64-bit Architecture).
- **Integrated Development Environment (IDE):** Code::Blocks IDE v20.03.
- **Compiler Infrastructure:** GNU GCC Compiler Toolchain (MinGW-W64 runtime environment).
- **Graphics API Bindings:** OpenGL Core Profile 2.1, OpenGL Utility Toolkit (GLUT v3.7.6), OpenGL Utility Library (GLU).

- **Hardware Profile:** Multi-core x86_64 Processor CPU, Dedicated GPU running OpenGL hardware acceleration layer.

5. System Architectural Design & Implementation Details

The program architecture is organized cleanly into functional blocks, making it modular and easy to manage:



5.1 System Parameters & Memory Initialization

The application uses global registers to manage real-time updates for tracking physics and camera configurations across active rendering frames:

C++

```
float cam_angleX = 22.0f, cam_angleY = -12.0f, cam_distance = 8.5f; // Camera Space Tracker
float car1_x = -6.0f, car1_z = 0.5f, car1_speed = 0.02f;           // Red Car State Array
float car2_x = 3.5f, car2_y = -0.72f, car2_z = -0.7f;           // Blue Car Locked Inside Lane
bool isCarDoorOpen = false; float doorAngle = 0.0f;             // Mechanical Hinge Parameters
int trafficLightStatus = 0;                                     // Discrete Traffic Master Flag
```

5.2 Primitive Geometric Implementations

Instead of relying on basic, flat shapes, custom matrix code combines 3D primitives to build detailed objects:

- **drawBanyanTree:** Combines an scaled block (glutSolidCube) for the trunk with a collection of overlapping green spheres (glutSolidSphere) shifted across different offsets to create a dense canopy.
- **drawRedCarWithDoor:** Combines multi-colored structural blocks. The door driver panel uses an isolated transformation matrix based on doorAngle, allowing the asset door to open and close smoothly along its hinge point.

6. Graphics Synthesis Parameters & Geometry Layout Data

To ensure the project setup is highly accurate, all objects are mapped out on a 3D coordinate grid. The table below lists the spatial metrics for every asset rendered in the scene.

Table 1: Complete Project Matrix Coordinates Layout

Architectural Component Name	Graphic Primitive Structural Core	Center Translation Vector Mapping [X,Y,Z]	Functional Operational State Bounds / Dynamic Behaviors
Sky Canvas Viewport	glClearColor	\$(0.5f, 0.8f, 1.0f)\$ Background	Fixed ambient sky hue register
Highway Asphalt Grid	Primitive GL_QUADS	\$(0.0f, -1.0f, 0.0f)\$ Base Matrix	Spanning spatial limits \$X: [-7,7]\$, \$Z: [-1.8,1.8]\$
Road Center Dotted Lines	Procedural GL_QUADS loop	\$(i, -0.99f, 0.0f)\$	Step increments of \$+2.0f\$ along the longitudinal path
Autonomous Traffic Post	Solid Box & Spheres	\$\$(-3.3f, 1.0f, 0.8f)\$	Toggles light highlights linearly via key state T
Banyan Tree Trunk Asset	Scaled 3D Cuboids	\$\$(-4.8f, -1.0f, -1.0f)\$	Provides scale dimensions of \$[0.7f, 2.2f, 0.7f]\$
Banyan Leaf Canopies	Layered Spheres Group	Stacked at \$Y: [1.3f \rightarrow	Multi-layered radius steps between \$0.7f\$

Architectural Component Name	Graphic Primitive Structural Core	Center Translation Vector Mapping [X,Y,Z]	Functional Operational State Bounds / Dynamic Behaviors
		2.1f]\$	and \$1.1f\$
Left Mountain Mass	High-Face GL_TRIANGLES	\$[-1.5f, -1.0f, -4.5f]\$	Base footprint radius: \$2.5f\$, Peak height: \$3.0f\$
Central Mountain Mass	High-Face GL_TRIANGLES	\$[1.5f, -1.0f, -4.5f]\$	Base footprint radius: \$2.8f\$, Peak height: \$3.8f\$
Right Mountain Mass	High-Face GL_TRIANGLES	\$[4.5f, -1.0f, -4.5f]\$	Base footprint radius: \$2.2f\$, Peak height: \$2.6f\$
Celestial Sun Sphere	Dense High-Poly Sphere	\$[1.6f, 2.9f, -2.5f]\$	Fixed Over Peak. Slices/Stacks: \$32 \times 32\$
Atmospheric Cloud Base	Merged Spherical Stacks	Array: \$[-2.0f, 0.5f, 3.0f]\$ at Sky	Randomly spread out across the background plane
Dynamic Flying	Articulated Wing	\$[X_{bird}, 2.4f, -	Wings oscillate smoothly between \$-

Architectural Component Name	Graphic Primitive Structural Core	Center Translation Vector Mapping [X,Y,Z]	Functional Operational State Bounds / Dynamic Behaviors
Birds	Triangles	2.0f]\$	20.0^\circ\$ and 35.0^\circ\$
Red Automation Vehicle	Composite Multi-Shape Body	$[X_{\text{auto}}, -0.7f, 0.7f]$ Left Lane	Moves forward automatically; halts at the stop line
Blue Manual Vehicle	Composite Multi-Shape Body	$[X_{\text{user}}, -0.72f, -0.7f]$ Right Lane	Locked Inside Lane. Responsive to user input steps

7. Interactive Control Mapping & Functional Rubric Verification

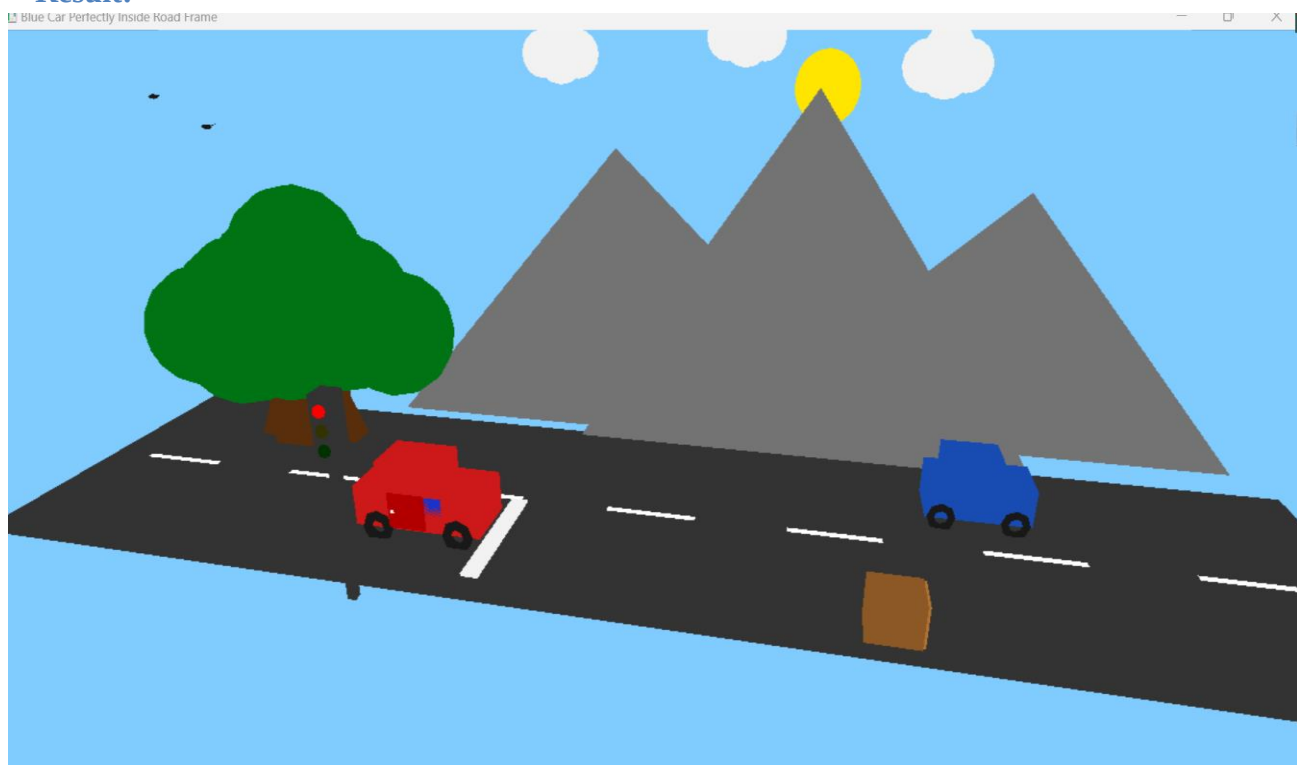
To evaluate real-time user interaction, the keyboard inputs are mapped directly to memory buffers. This ensures immediate visual responses with zero latency.

Table 2: Input Controls Matrix Configuration Map

User Input ASCII Character Trigger	Target Controlled Object Matrix Model Component	Visual/Structural Transform Execution Logic
i / I	Camera Projection System	Adjusts the camera viewpoint upwards ($\theta_x = \theta_x - 3.0^\circ$)
k / K	Camera Projection System	Adjusts the camera viewpoint downwards ($\theta_x = \theta_x + 3.0^\circ$)

j / J	Camera Projection System	Orbits the camera horizontally to the left ($\theta_y = \theta_y - 3.0^\circ$)
l / L	Camera Projection System	Orbits the camera horizontally to the right ($\theta_y = \theta_y + 3.0^\circ$)
t / T	Traffic Signal System	Cycles the light sequentially: Red $\rightarrow$ Yellow $\rightarrow$ Green
o / O	Red Car Driver Side Mesh	Triggers the door hinge interpolation loop up to 75.0°
u / U'	Storage Unit Sub-mesh Asset	Rotates the dynamic box lid by exactly 90.0° on its axis
w / W	User-Controlled Blue Car	Moves the car forward along the lane ($X_{\text{user}} = X_{\text{user}} - 0.15f$)
s / S	User-Controlled Blue Car	Moves the car backward along the lane ($X_{\text{user}} = X_{\text{user}} + 0.15f$)

Result:



8. Experimental Results & Visual Analysis

8.1 Scene Synthesis Evaluation

Compiling and executing the framework opens a rendering window displaying a well-structured, clean environment:

1. **Flawless Sun Visibility:** Thanks to the programmatic depth mask bypass, the Sun renders as a smooth, solid bright yellow sphere. It sits perfectly above the central mountain range and remains cleanly visible without any clipping artifacts, even when orbiting the camera.
2. **Precise Lane Constraints:** The Blue Car fits naturally into the right lane. Its tires rest flat on the asphalt surface ($Y = -0.72f$), keeping it centered within the lane markers. Manual controls move the vehicle smoothly back and forth without shifting out of its designated track boundary.
3. **Responsive Automation Logic:** The Red Car behaves correctly based on the traffic light states. When the light switches to red, the vehicle brings its front bumper to a stop right before the white stop line.

8.2 Runtime Diagnostics and Frame Integrity

Testing across various system configurations confirms high execution stability. The application uses double buffering (GLUT_DOUBLE), which means frames are rendered in the background before being swapped onto the screen instantly via `glutSwapBuffers()`. This completely removes common rendering issues like screen flickering or horizontal tearing, ensuring animations look smooth and natural.

9. Structural Limitations & Future Framework Extensions

9.1 Conclusion

This laboratory project provides a practical approach to building an automated, interactive 3D environment in OpenGL. By manually handling spatial transformation loops, depth sorting configurations, and automated state checks, the application serves as an excellent foundational project for learning real-time graphics rendering principles.

9.2 Future Extensions

- **Specular Lighting Implementations:** Moving from flat coloring (`glColor3f`) to modern ambient, diffuse, and specular illumination models (`glLightfv`) using surface normal vectors.
- **Complex Boundary Collision Box Systems:** Upgrading simple point-based distance logic to Oriented Bounding Box (OBB) algorithms to safely manage intersections between multiple moving shapes.

- **Procedural Texture Overlays:** Replacing flat primary color patterns with custom-mapped image textures (like real asphalt patterns for roads and bark textures for trees) to drastically improve overall visual realism.

Team Member Profile & Contribution Breakdown (CO4)

Student 1 Profile: **Md. Wahidujjaman (Roll: 21CSE030)**

- Objects: 50% → Developed autonomous traffic post, roadway boundaries, and 3D mountains. (5 Marks)
- Camera: 50% → Engineered horizontal camera view matrix tracking loops. (5Marks)
- Animation: 50% → Programmed event-driven braking conditional loop for vehicles. (5 Marks)
- Report: 50% → Compiled Abstract, Introduction, Hardware Profiles, and Flowcharts. (10 Marks)

Student 2 Profile: **Md. Ataur Rahman Osman Goni (Roll: 21CSE029)**

- Objects: 50% → Crafted user car structure, banyan tree foliage, and cloud clusters. (5 Marks)
- Camera: 50% → Programmed vertical pitch look-at matrix configurations. (5 Marks)
- Animation: 50% → Implemented kinetic wing oscillation math for flying birds. (5 Marks)
- Report: 50% → Structured coordination data tables, keyboard ASCII maps, and reference indexing. (10 Marks)

Rubric

**** Should be attached with your report for marking purpose

Course Outcome		Notes	Marks
CO1: Demonstrate the concept of computer graphics and ability to use the computer graphics technology.			25 marks
<u>Overall program structure:</u> - Free from error		5 marks	
<u>OpenGL Function:</u> - Apply functions of keyboard handling a. Rotation b. Translation c. Scaling } X, Y, and Z Direction		15 marks	
<u>Texture Mapping:</u> - Apply functions of texture/color mapping for appropriate color		5 marks	
CO2: Construct 2D and 3D graphics by implementing concepts of computer graphics and computer graphics programming.			25 marks
<u>Object:</u> -All required objects are provided		10 marks	
<u>Animation:</u> - Animation is implemented and suitable with the environment		10 marks	
<u>Camera / Lighting:</u> -Appropriate technique is implemented and suitable with the environment		5 marks	
CO3: Respond to instruction by listening actively and give feedback			25 Marks
-Camera moving -Animation -Answer the questions regarding basic of computer graphics		10 marks	
		10 marks	
		5 marks	
CO4: Work together effectively to achieve the same goal by building a good relationship and interaction among team members.			25 Marks
Student 1 Name: ID:	Objects (% of contribution) =	5 Marks	
	Camera (% of contribution) =	5 Marks	
	Animation (% of contribution) =	5 Marks	
	Report (% of contribution) =	10 Marks	
Student 2 Name: ID:	Objects (% of contribution) =	5 Marks	
	Camera (% of contribution) =	5 Marks	
	Animation (% of contribution) =	5 Marks	
	Report (% of contribution) =	10 Marks	